

DATABASE MANAGEMENT SYSTEM

LABORATORY EQUIPMENT BOOKING & MAINTENANCE MANAGEMENT SYSTEM

PROJECT REPORT

Submitted by

Shaurya Dhingra

Roll Number: 25WU0102254

Programme/Section: AIML Whales

Project Title

Design and Implementation of a Database Management System for Laboratory Equipment
Booking and Maintenance

Course: Database Management Systems (DBMS)

Faculty: Dr. Kiran Mayee

Academic Year: 2026–2027

CONTENTS

1. Abstract
2. Introduction and Problem Statement
3. Objectives and Scope
4. Software and Hardware Requirements
5. ER Diagram and Relationships
6. Relational Schema and Normalization
7. Data Dictionary
8. SQL Commands (DDL and DML)
9. Queries and Sample Outputs
10. User Interface and Screenshots
11. Implementation Details
12. Testing
13. Conclusion and Future Enhancements
14. References
15. Appendix: GitHub Repository

1. ABSTRACT

Keeping track of laboratory equipment can become confusing when bookings, equipment details, and maintenance records are stored in separate files or registers. This project brings those records together in one relational database so that users can find information more easily and staff can manage bookings with fewer manual checks.

The system uses MySQL to store the data and a Python Flask application to provide a browser-based interface. The database is organised into ten related tables: USER, LABORATORY, EQUIPMENT_CATEGORY, EQUIPMENT, BOOKING, TECHNICIAN, MAINTENANCE_TYPE, MAINTENANCE, FEEDBACK, and PAYMENT. Primary keys identify records, while foreign keys connect related records.

The interface supports viewing, searching, inserting, and deleting records. It also includes selected validation rules and SQL reports. In the demonstrated booking workflow, the booking count increased from 30 to 31 after an insertion, the new row was checked in MySQL, and deleting it brought the count back to 30. This is evidence that the demonstrated UI actions were reflected in the database.

Keywords: DBMS, MySQL, Flask, laboratory equipment, booking, maintenance, referential integrity.

2. INTRODUCTION AND PROBLEM STATEMENT

Laboratories use different types of equipment for practical work, experiments, and research. Each item needs to be available when required and maintained when it develops a fault. If the details are kept in paper registers or separate spreadsheets, it can take extra time to check availability, find a past booking, or work out when equipment was last serviced.

Booking conflicts are another concern. Two people may request the same item for overlapping periods, or someone may try to book equipment that is damaged or being repaired. A connected database can help keep these records consistent and make it easier to apply checks before a booking is accepted.

Problem statement. The aim is to build a database system that keeps laboratory, equipment, booking, and maintenance information organised, protects relationships between records, and demonstrates database operations through a working interface connected to MySQL.

Proposed solution. MySQL stores the records, while Flask provides pages and forms in a web browser. The database layer runs SQL statements and returns results to the interface. The app can show a success message and updated row count after an insertion or deletion.

3. OBJECTIVES AND SCOPE

Objectives

- Keep information for the ten main entities in separate, related tables.
- Use primary keys and foreign keys to identify records and maintain relationships.
- Allow users to view, search, insert, and delete records through the interface.
- Check common form inputs, including dates, times, email addresses, phone numbers, and numeric values.
- Reject selected invalid bookings, such as bookings for damaged or maintained equipment and overlapping confirmed/completed slots.
- Provide SQL reports for bookings, equipment usage, maintenance, payments, and feedback.

Scope and limitations

The current prototype covers the database named *laboratory_management*, a local Flask interface, view/insert/delete operations, selected validation rules, and SQL reports. Payment is stored as a record; an online payment gateway is not included. Login roles, edit/update workflows, email reminders, and mobile support are possible future additions. The project is a course prototype, not a production service.

4. SOFTWARE AND HARDWARE REQUIREMENTS

Category	Requirement
Operating system	Windows, Linux, or macOS
Programming language	Python 3.9 or later
Web framework	Flask 3.x or compatible
Database	MySQL Server; database name: laboratory_management
Database driver	PyMySQL (the database layer also describes mysql-connector support)
Browser	A modern browser such as Chrome, Edge, or Firefox
Python dependencies	flask and pymysql, listed in requirements.txt
Hardware	A laptop or desktop able to run Python, MySQL Server, and a browser

The database connection settings are read from config.py and environment variables. The supplied configuration uses localhost, port 3306, the root user, and the database name laboratory_management by default. The password value is an example setting, so it must be changed to match the local MySQL installation. A real password should not be uploaded to a public repository.

Typical setup commands

```
pip install -r requirements.txt
# Set DB_PASSWORD / DB_USER / DB_HOST if needed
python app.py
# Open http://127.0.0.1:5000
```

5. ER DIAGRAM AND RELATIONSHIPS

The database design has ten entities. A user can make bookings, and each booking refers to a user and a piece of equipment. Equipment belongs to a laboratory and an equipment category. Maintenance records connect equipment with a technician and a maintenance type. Feedback and payment records are linked to bookings.

Parent table	Related table	Relationship purpose
USER	BOOKING	Shows who requested the equipment
EQUIPMENT	BOOKING	Shows which item was requested
LABORATORY	EQUIPMENT	Records where equipment is located
EQUIPMENT_CATEGORY	EQUIPMENT	Groups similar equipment
EQUIPMENT	MAINTENANCE	Keeps servicing history for an item
TECHNICIAN	MAINTENANCE	Identifies who performed maintenance
MAINTENANCE_TYPE	MAINTENANCE	Classifies the work carried out
BOOKING	FEEDBACK	Links feedback to a booking
BOOKING	PAYMENT	Links payment details to a booking

Use the original ER diagram from Presentation-II in the final repository/report if the course requires the exact diagram submitted earlier. This section explains the relationships but does not claim to reproduce that original image.

6. RELATIONAL SCHEMA AND NORMALIZATION

Table	Primary key	Main role / references
USER	user_id	User details such as name, email, phone, type, and department
LABORATORY	lab_id	Lab name, location, capacity, and status
EQUIPMENT_CATEGORY	category_id	Equipment category name and description
EQUIPMENT	equipment_id	References EQUIPMENT_CATEGORY and LABORATORY
BOOKING	booking_id	References USER and EQUIPMENT
TECHNICIAN	technician_id	Technician contact details and specialisation
MAINTENANCE_TYPE	maintenance_type_id	Types of maintenance work
MAINTENANCE	maintenance_id	References EQUIPMENT, TECHNICIAN, and MAINTENANCE_TYPE
FEEDBACK	feedback_id	References BOOKING and USER
PAYMENT	payment_id	References BOOKING

Normalization

The design separates different kinds of information instead of placing everything in one large table. For example, equipment stores IDs for its laboratory and category rather than repeating all of those details in every booking. Maintenance uses references to equipment, technicians, and maintenance types, while feedback and payment are stored separately from booking details. This reduces unnecessary repetition and makes relationships easier to maintain.

At a basic level, the tables are intended to meet First Normal Form because fields hold individual values. Each table has a single-column primary key, which supports the intended Second Normal Form design. Related descriptive details are separated into their own tables, supporting a Third Normal Form-style structure. A formal proof would require checking the complete SQL schema and functional dependencies.

7. DATA DICTIONARY

This summary follows the fields configured for the interface in `schema_config.py`. The final MySQL CREATE TABLE statements determine the exact SQL types, defaults, and constraints.

Table	Main fields	Notes
USER	user_id, name, email, phone, user_type, department	Primary key; email and phone validation in UI
LABORATORY	lab_id, lab_name, location, capacity, lab_status	Capacity should be at least 1
EQUIPMENT_CATEGORY	category_id, category_name, description	Groups equipment by category
EQUIPMENT	equipment_id, equipment_name, category_id, lab_id, serial_number, purchase_date, equipment_status, condition, quantity	References category and lab; quantity should be at least 1
BOOKING	booking_id, user_id, equipment_id, booking_date, start_time, end_time, purpose, booking_status	References user and equipment; time interval and overlap checks
TECHNICIAN	technician_id, technician_name, email, phone, specialization	Contact and specialisation details
MAINTENANCE_TYPE	maintenance_type_id, type_name, description	Classifies maintenance work

Table	Main fields	Notes
MAINTENANCE	maintenance_id, equipment_id, technician_id, maintenance_type_id, maintenance_date, description, cost, maintenance_status, next_maintenance_date	References equipment, technician, and type
FEEDBACK	feedback_id, booking_id, user_id, rating, comments, feedback_date	Rating is configured from 1 to 5
PAYMENT	payment_id, booking_id, amount, payment_date, payment_method, payment_status, transaction_reference	Amount must be greater than zero in the app

PK means primary key and FK means foreign key. The database schema is the final authority for actual NOT NULL, UNIQUE, CHECK, DEFAULT, and foreign-key rules.

8. SQL COMMANDS (DDL AND DML)

Data Definition Language (DDL) is used to create database objects, while Data Manipulation Language (DML) is used to read or change the records. The short example below shows the general pattern. It is not a replacement for the complete CREATE TABLE script submitted for Presentation-II.

```
CREATE DATABASE IF NOT EXISTS laboratory_management;
USE laboratory_management;

CREATE TABLE LABORATORY (
    lab_id INT PRIMARY KEY AUTO_INCREMENT,
    lab_name VARCHAR(100) NOT NULL,
    location VARCHAR(100) NOT NULL,
    capacity INT NOT NULL,
    lab_status VARCHAR(30) DEFAULT 'Active'
);
```

The following commands were used during the live demonstration:

```
SELECT COUNT(*) AS total_equipment FROM equipment;
SELECT COUNT(*) AS total_bookings FROM booking;

SELECT *
FROM booking
ORDER BY booking_id DESC
LIMIT 1;

SELECT * FROM booking WHERE booking_id = 31;
DELETE FROM booking WHERE booking_id = 31;
```

Observed results from the supplied screenshots

Check	Observed result
Equipment row count	30
Booking row count before insertion	30
Insert a booking	Booking ID 31 created; count changed from 30 to 31
Verify booking ID 31	MySQL displayed user_id 1, equipment_id 1, date 2026-10-07, 13:00–20:00, purpose 'Nothing', status Confirmed
Delete booking ID 31	Deletion succeeded; booking count returned from 31 to 30
Counts for all ten tables	30 rows in each table after cleanup, as shown in the supplied MySQL screenshot

9. QUERIES AND SAMPLE OUTPUTS

The Presentation-III materials describe reports that combine related records and summarise equipment usage. The examples below show the type of SQL used. The actual rows returned depend on the database contents at the time the query is run.

Query 1 — Booking details

```
SELECT b.booking_id, u.name AS booked_by, e.equipment_name,
       l.lab_name, b.booking_date, b.start_time, b.end_time,
       b.booking_status
FROM BOOKING b
JOIN `USER` u ON u.user_id = b.user_id
JOIN EQUIPMENT e ON e.equipment_id = b.equipment_id
JOIN LABORATORY l ON l.lab_id = e.lab_id
ORDER BY b.booking_date DESC
LIMIT 8;
```

This query joins four tables so that the booking list includes the user, equipment, and laboratory names instead of showing only IDs.

Query 2 — Equipment utilisation

```
SELECT e.equipment_name, COUNT(b.booking_id) AS bookings
FROM EQUIPMENT e
JOIN BOOKING b ON b.equipment_id = e.equipment_id
WHERE b.booking_status NOT IN ('Cancelled', 'Rejected')
GROUP BY e.equipment_name
ORDER BY bookings DESC, e.equipment_name
LIMIT 8;
```

This query counts bookings for each item while leaving cancelled and rejected bookings out of the total.

Query 3 — Equipment needing attention

```
SELECT e.equipment_name, l.lab_name,
       e.equipment_status, e.`condition`
FROM EQUIPMENT e
JOIN LABORATORY l ON l.lab_id = e.lab_id
WHERE e.equipment_status IN ('Under Maintenance', 'Damaged');
```

10. USER INTERFACE AND SCREENSHOTS

The application runs in a browser and is built with Flask. A sidebar gives access to the dashboard and the ten database tables. The table pages support viewing and searching records, opening an insertion form, and deleting a record after confirmation. Where possible, foreign-key IDs are shown with the related names, and status values use colours to make them easier to scan.

The dashboard is intended to summarise equipment and booking activity, including availability, pending bookings, maintenance due dates, payments, and table counts. The Reports page groups information such as booking details, equipment utilisation, overdue maintenance, equipment needing attention, payment summaries, and feedback ratings.

During insertion, the app shows a success message and the row count before and after the change. The screenshot evidence supplied for this report shows a booking inserted as ID 31, the matching row in MySQL, and a successful deletion that returned the booking count to 30. Foreign-key relationships can prevent a parent row from being deleted while dependent records still refer to it.

Screenshot to include	What it should demonstrate
Dashboard	Summary cards, table navigation, and activity information
Booking list before insert	30 bookings before the test insertion
Insert success	Booking #31 inserted and count changed 30 → 31
MySQL SELECT output	Booking #31 and its saved values
Delete success	Booking #31 deleted and count changed 31 → 30
Equipment list / MySQL count	30 equipment records and equipment status values
SQL reports page	Examples using joins, grouping, or aggregate functions

The screenshots folder has been uploaded under Presentation-III in the GitHub repository. Keep the original screenshots there. If the faculty expects images embedded in the report itself, insert the relevant screenshots into this section before submitting the final PDF.

11. IMPLEMENTATION DETAILS

File or component	What it does
app.py	Flask routes for dashboard, table view/search, insertion, deletion, reports, and validation
db.py	Database connection and SQL execution; includes MySQL and SQLite demo-backend support
config.py	Database host, port, user, password, database name, and backend settings
schema_config.py	Defines ten table forms, field types, dropdown references, and unbookable statuses
seed_data.py	Sample records for the optional SQLite demo database
demo_db.py	Creates a SQLite demo copy so the interface can be tried without MySQL
templates/ and static/	HTML pages and CSS styles
capture_screenshots.py	Automates browser scenarios using Playwright
requirements.txt	Lists the Python packages needed by the app

Database connectivity

The Flask routes use the database layer to run SQL statements. Form values are passed as parameters, which helps reduce SQL injection risk. After an insert or delete, the app checks the current row count and shows the change. Foreign keys help preserve the links between tables and may reject deletion of a record that is still referenced elsewhere.

Configured business rules

- End time must be later than start time.
- Equipment marked Under Maintenance or Damaged cannot be booked through the configured booking flow.
- Confirmed/completed bookings are checked for overlapping time slots for the same equipment and date.
- The next maintenance date must be later than the maintenance date.
- Payment amount must be greater than zero.
- Required fields and selected email, phone, date, time, numeric, and choice fields receive form validation.

12. TESTING

The table below separates checks that are visible in the supplied screenshots from other cases listed in the project test plan. This distinction matters because a planned test should not be described as passed until its result has been checked.

Test case	Expected result	Evidence status
Open Bookings table	Records and row count appear	Shown in UI screenshot
Count equipment	COUNT(*) returns 30	Shown in MySQL screenshot
Count bookings before insert	COUNT(*) returns 30	Shown in MySQL screenshot
Insert a booking	UI shows 30 → 31	Shown in UI screenshot
Verify inserted booking	MySQL displays booking_id 31	Shown in MySQL screenshot
Delete inserted booking	UI shows 31 → 30	Shown in UI screenshot
Count all ten tables	30 rows in each after cleanup	Shown in MySQL screenshot
Overlapping confirmed booking	Rejected with a time-conflict message	Listed in test plan; rerun for final evidence
Book equipment under maintenance	Rejected	Listed in test plan; rerun for final evidence
End time before start time	Form rejects invalid interval	Listed in test plan; rerun for final evidence
Invalid email or phone	Form validation rejects input	Listed in test plan; rerun for final evidence
Duplicate email or serial number	Uniqueness rule rejects duplicate	Check actual DDL and rerun
Delete a referenced parent record	Foreign key blocks deletion	Rerun and capture exact message

Verified result summary

The clearest verified workflow is the insert-and-delete test. The booking count started at 30, increased to 31 after the UI insertion, and the MySQL console showed the new row. After the test booking was deleted, the count returned to 30. The supplied console screenshot also showed 30 rows in each of the ten tables after cleanup. The other validation cases should be rerun and recorded before the report is treated as final.

13. CONCLUSION AND FUTURE ENHANCEMENTS

This project brings laboratory equipment, booking, and maintenance records into a relational database and provides a browser interface for managing them. Splitting the information into ten tables keeps different types of data organised, while primary and foreign keys help maintain the links between records. The UI demonstrates common database tasks such as viewing, searching, inserting, and deleting records.

The live demonstration showed that inserting and deleting a booking through the interface changed the connected MySQL database as expected. There is still room to improve the system before it could be used in a real laboratory. Useful next steps would be adding login and role-based access, allowing records to be edited, introducing a booking approval process, sending maintenance reminders, and adding usage analytics. A payment gateway or QR-code check-in system could also be considered if the project is expanded.

14. REFERENCES

1. DBMS Course Project – Final Presentation Instructions, AIML Whales and AIML Panthers, course handout, 2026.
2. Project 29 review material: Laboratory Equipment Booking and Maintenance System, Presentation-I review deck.
3. DBMS Presentation 2, database design and query material supplied for the course project.
4. Presentation III, material on the Flask UI and MySQL connectivity.
5. Project source files: app.py, config.py, db.py, schema_config.py, demo_db.py, seed_data.py, requirements.txt, README.md, and capture_screenshots.py.
6. Flask documentation: <https://flask.palletsprojects.com/>
7. MySQL documentation: <https://dev.mysql.com/doc/>
8. PyMySQL documentation: <https://pymysql.readthedocs.io/>

15. APPENDIX: GITHUB REPOSITORY

Repository: DBMS-Course-Project

URL: <https://github.com/ShauryaDhingra001/DBMS-Course-Project>

```
DBMS-Course-Project/  
  Presentation-I/  
  Presentation-II/  
DBMS Presentation 2.pdf  
  README.md  
  Database SS/  
  Presentation-III/  
Presentation 3.pptx  
Presentation 3.pdf  
  README.md  
  app.py, db.py, config.py  
  schema_config.py, seed_data.py, demo_db.py  
  requirements.txt, capture_screenshots.py  
  screenshots/  
Project-Report/  
  README.md  
  Laboratory_Equipment_Booking_Maintenance_Project_Report.pdf
```